

UNSUPERVISED
LEARNING
CLUSTERING

Fundamentals of Machine Learning

Agenda

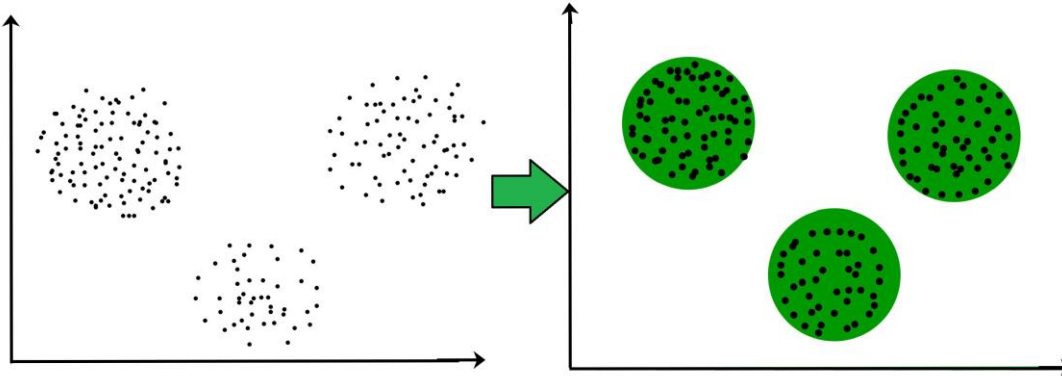
Introduction to clustering

k-Means

Introduction to clustering

What is clustering?

- The organization of unlabeled data into similarity groups called clusters.
- A cluster is a collection of data items which are “similar” between them, and “dissim



Introduction to clustering

Clustering vs Classification

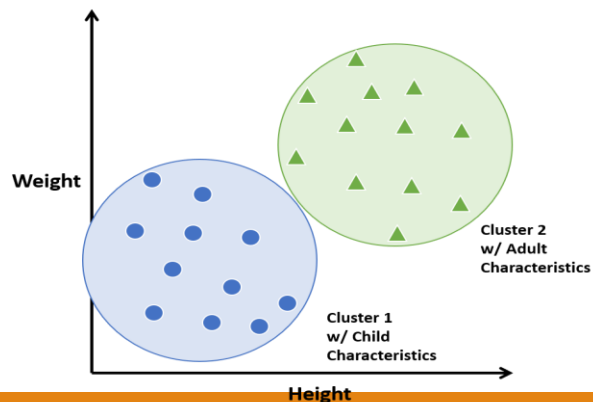
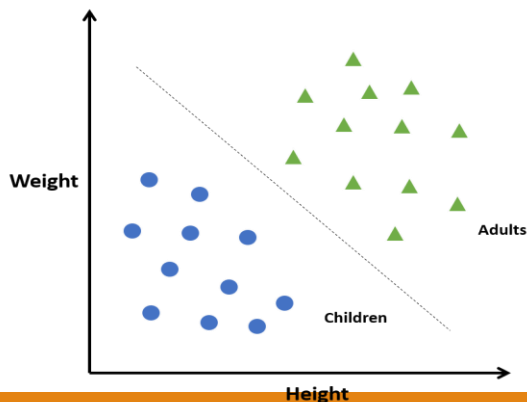
Supervised Classification

- Known number of classes
- Based on a training set
- Used to classify future observation

Unsupervised Clustering

- Unknown number of classes
- No prior knowledge
- Used to understand (explore) data

Classification vs Clustering



Introduction to clustering

Applications of Clustering

- **Retail/marketing:**
 - Identify customers with similar behavior
 - Recommending new product to new customer
- **Banking:**
 - Fraud detection in credit card use
 - Identify clusters of customers
- **Insurance:**
 - Fraud detection in claims analysis
 - Insurance risk of customers

Introduction to clustering

Applications of Clustering

- **Publication:**
 - Auto-categorizing news based on their content
- **Medicine:**
 - Characterizing patient behavior
- **Biology:**
 - Clustering genetic markers to identify family ties

Introduction to clustering

Why clustering?

- Exploratory data analysis
- Summary generation
- Outlier detection
- Finding duplicates
- Pre-processing step

Introduction to clustering

Clustering algorithms

- Partition-based clustering
 - Relatively efficient
 - e.g: k-Mean, k-Median, ...
- Hierarchical Clustering
 - Produces tree of clusters
 - e.g: Agglomerative, Divisive
- Density-based Clustering
 - Produces arbitrary shaped clusters
 - e.g: DBSCAN

Clustering

K-MEAN

K-Mean

What is k-Means clustering?

- K-means (MacQueen, 1967) is a **partitional clustering** algorithm
- Let the set of data points D be $\{x_1, x_2, \dots, x_n\}$, where $x_i = (x_{i_1}, x_{i_2}, \dots, x_{i_r})$ is a vector in $X \in \mathbb{R}^r$, and r is the number of dimensions.
- The k – *means* algorithm partitions the given data into k clusters:
 - Each cluster has a cluster center, called centroid.
 - k is specified by the user

K-Mean

K-means algorithm

Given k , the $k - means$ algorithm works as follows:

- 1. Choose k (random) data points (seeds) to be the initial centroids, **cluster** centers
- 2. Assign each data point to the closest **centroid**
- 3. Re-compute the **centroids** using the current cluster memberships
- 4. If a convergence criterion is not met, repeat steps 2 and 3

K-Mean

K-means convergence (stopping) criterion

- no (or minimum) re-assignments of data points to different clusters, or
- no (or minimum) change of centroids, or
- minimum decrease in the sum of squared error (SSE),

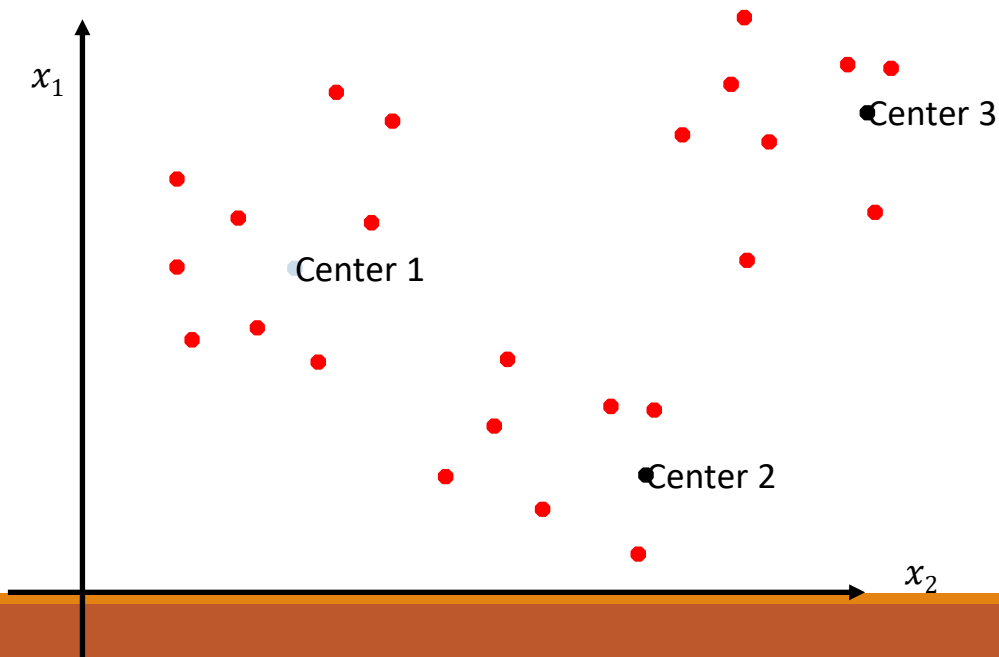
$$sse = \sum_{j=1}^k \sum_{x \in C_j} d(x, m_j)^2$$

- C_j is the j th cluster,
- m_j is the centroid of cluster C_j (the mean vector of all the data points in C_j),
- $d(x, m_j)$ is the (Euclidean) distance between data point x and centroid m_j .

K-Mean

K-means clustering example

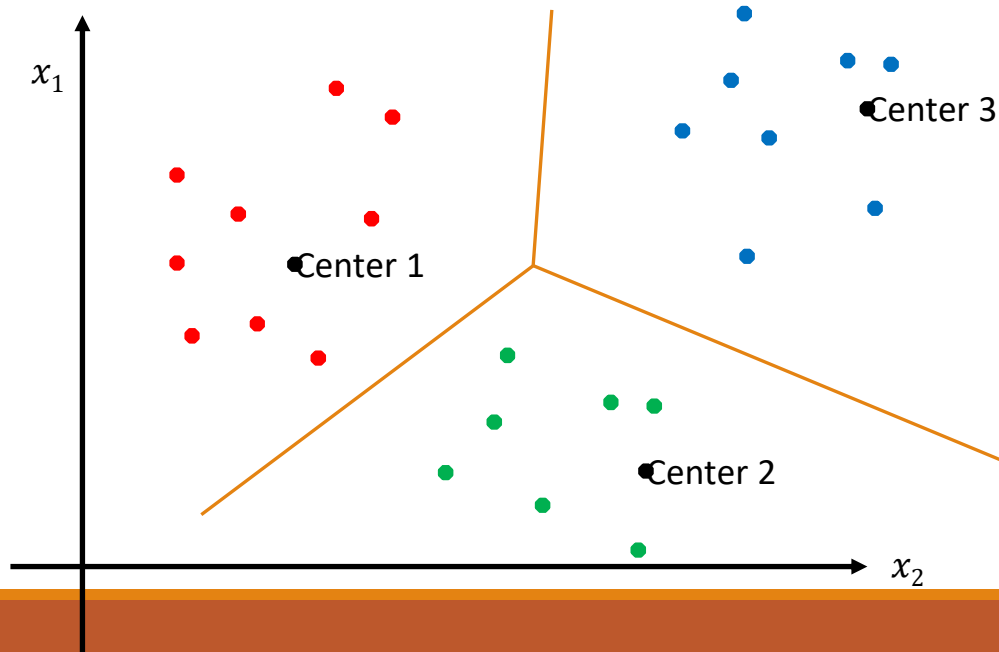
Step 1: Randomly initialize the cluster centers



K-Mean

K-means clustering example

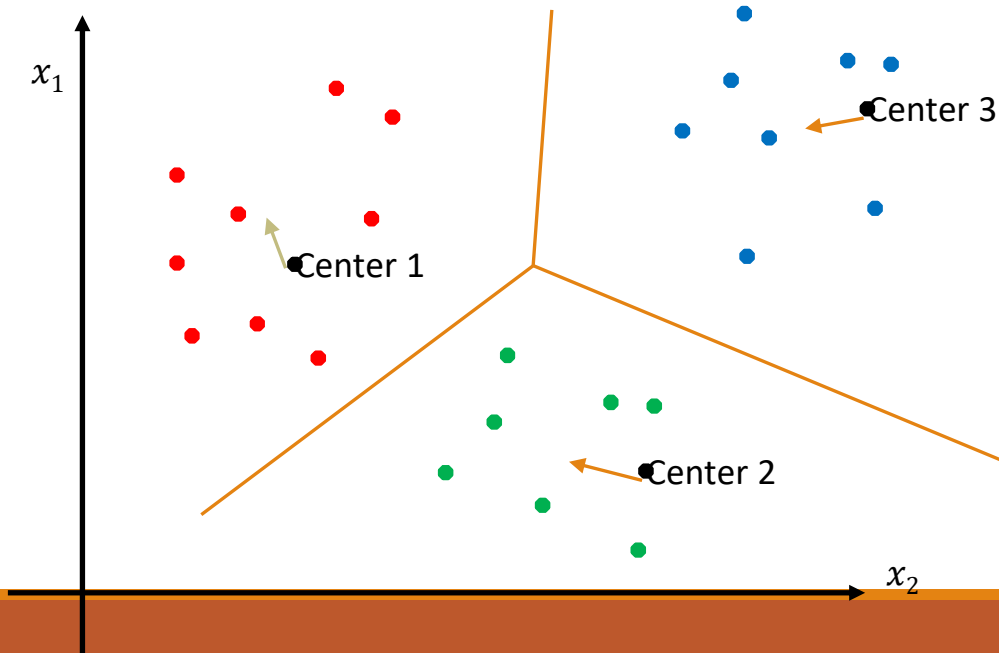
Step 2: Determine cluster membership for each input



K-Mean

K-means clustering example

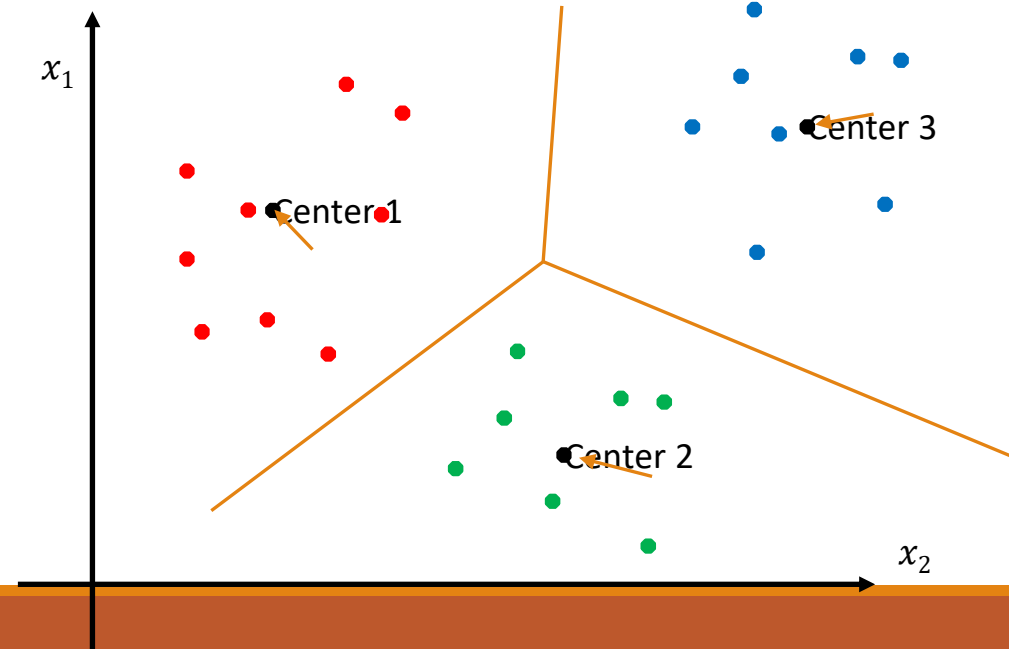
Step 3: Re-estimate cluster centers



K-Mean

K-means clustering example

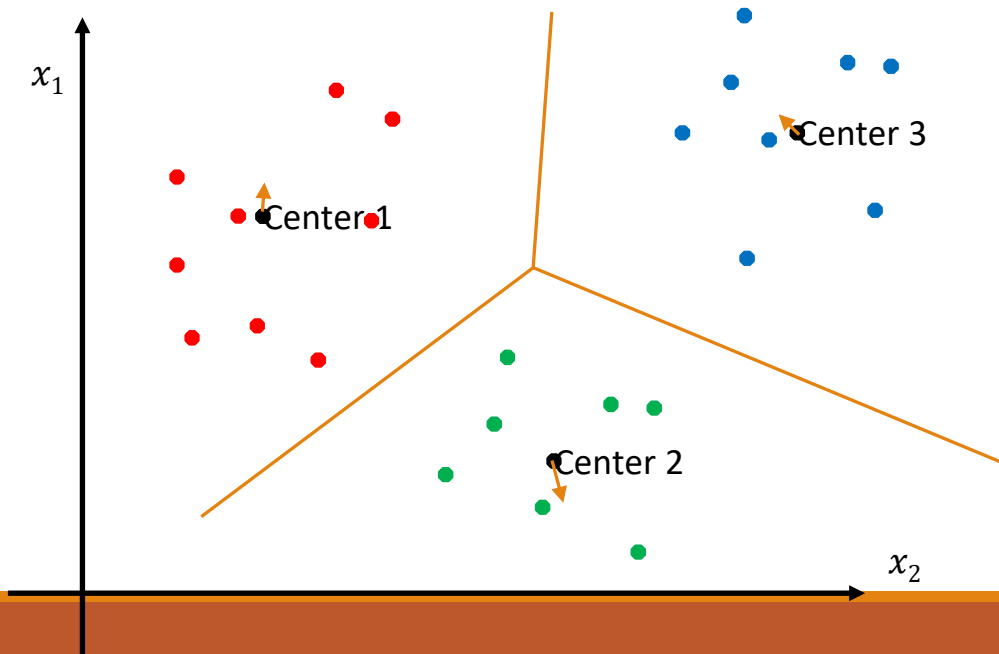
Result of first iteration



K-Mean

K-means clustering example

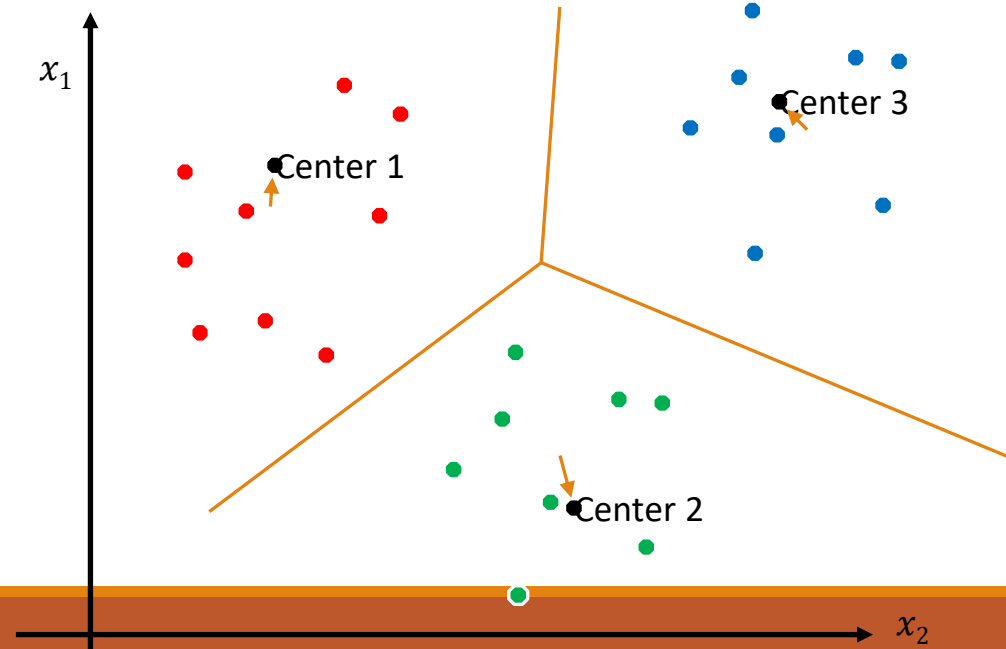
Second iteration



K-Mean

K-means clustering example

Result of second iteration



K-Mean

Why use K-means?

- Strengths:
 - Simple: easy to understand and to implement
 - Efficient: Time complexity: $O(tkn)$,
where n is the number of data points,
 - k is the number of clusters, and
 - t is the number of iterations.
 - Since both k and t are small. k – *means* is considered a linear algorithm.
- K-means is the most popular clustering algorithm.
- Note that: it terminates at a local optimum if SSE is used. The global optimum is hard to find due to complexity

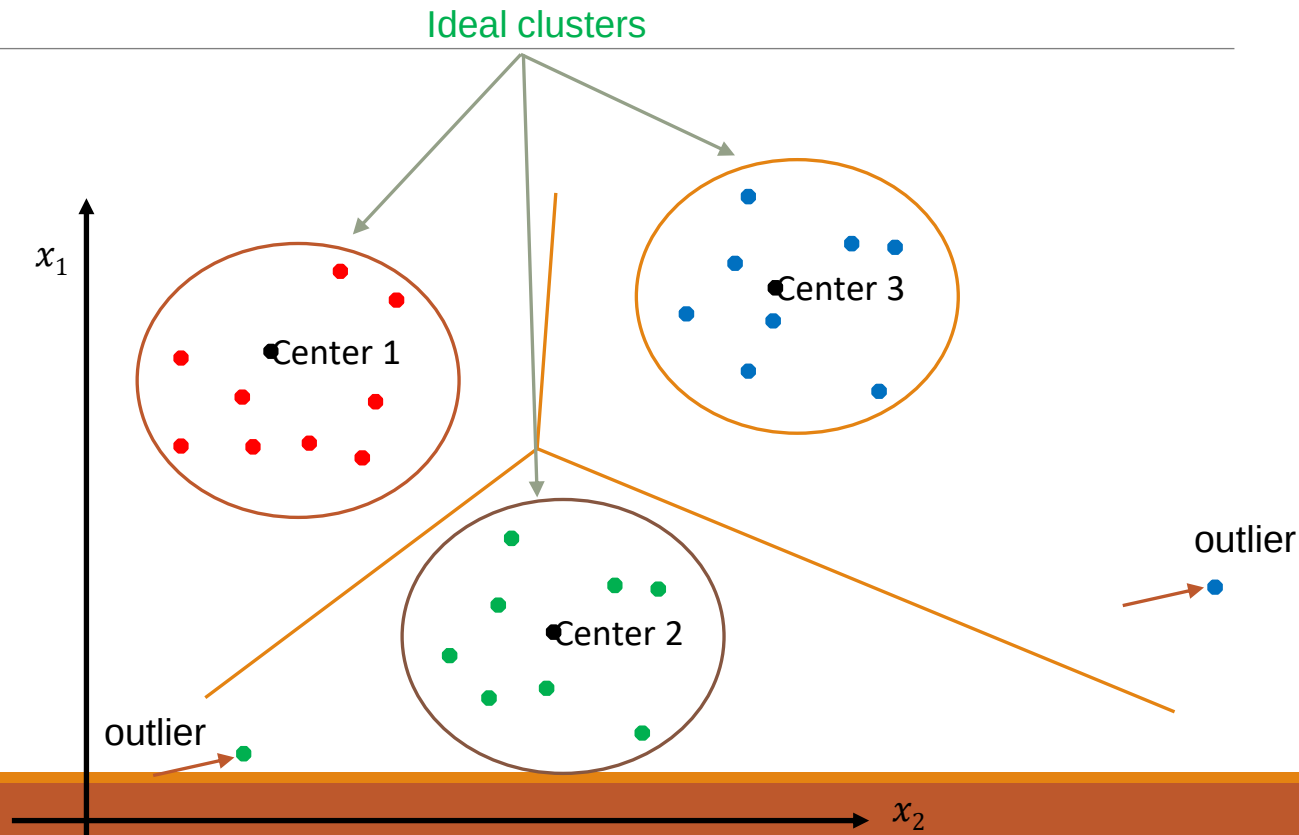
K-Mean

Weaknesses of K-means

- The algorithm is only applicable if the **mean** is defined.
 - For categorical data, k-mode - the centroid is represented by most frequent values.
- The user needs to specify ***k***.
- The algorithm is sensitive to **outliers**
 - Outliers are data points that are very far away from other data points.
 - Outliers could be errors in the data recording or some special data points with very different values.

K-Mean

Outliers



K-Mean

Dealing with outliers

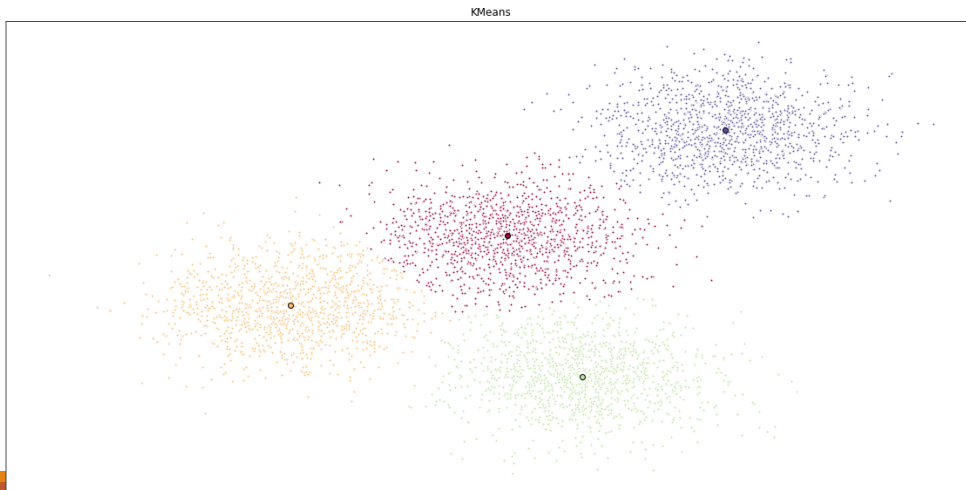
- Remove some data points that are much further away from the centroids than other data points
 - To be safe, we may want to monitor these possible outliers over a few iterations and then decide to remove them.
- Perform random sampling: by choosing a small subset of the data points, the chance of selecting an outlier is much smaller
 - Assign the rest of the data points to the clusters by distance or similarity comparison, or classification

K-Mean

K-Mean with sklearn

```
from sklearn.cluster import KMeans
from sklearn.datasets.samples_generator import make_blobs

X, y = make_blobs(n_samples=5000, centers=[[4,4], [-2, -1], [2, 3], [1, 1]], cluster_std=0.9)
k_means = KMeans(init = "k-means++", n_clusters = 4, n_init = 12)
k_means.fit(X)
```



K-Mean

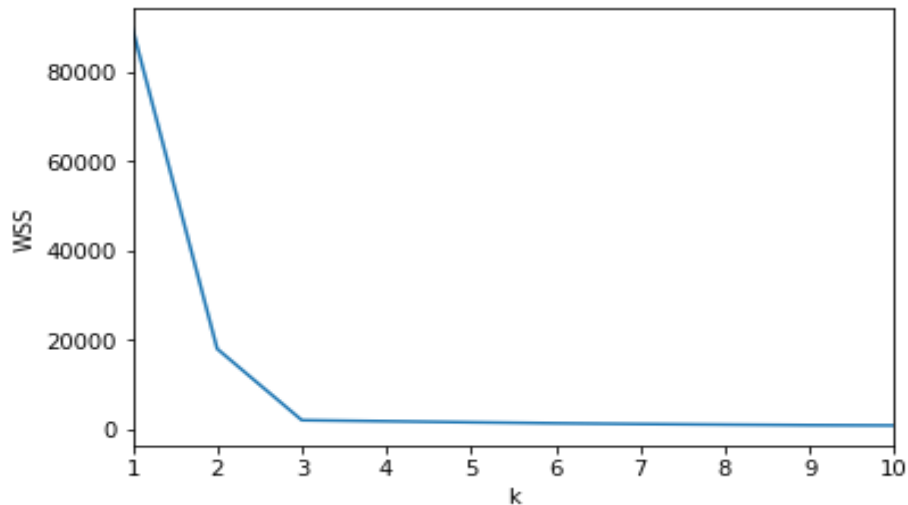
K-means summary

- Despite weaknesses, k-means is still the most popular algorithm due to its simplicity and efficiency
- No clear evidence that any other clustering algorithm performs better in general
- Comparing different clustering algorithms is a difficult task. No one knows the correct clusters!

K-Mean-Optimal K

- **Elbow Method:**

Calculate the **Within-Cluster-Sum of Squared Errors (WSS)** for **different values of k** , and choose the k for which WSS becomes first starts to diminish. In the plot of WSS-versus- k , this is visible as an **elbow**.



K-Mean-Optimal K

- **Silhouette Method**

The Silhouette Score reaches its *global maximum at the optimal k*. This should ideally appear as a peak in the Silhouette Value-versus-k plot.

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}, \text{ if } |C_i| > 1$$

and

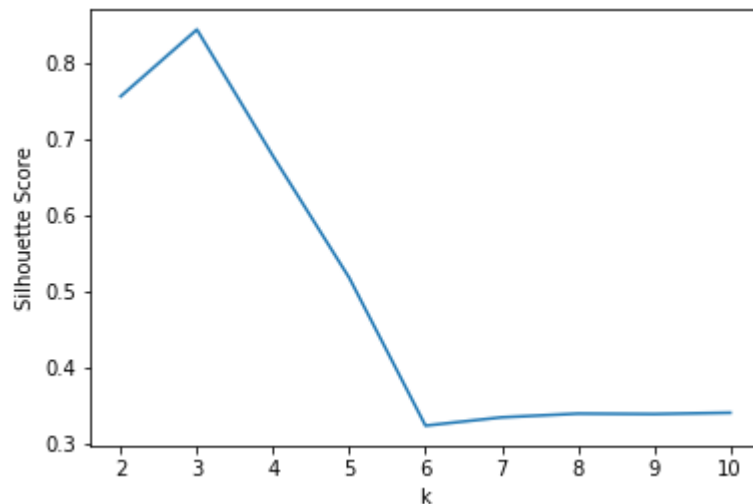
$$s(i) = 0, \text{ if } |C_i| = 1$$

For each data point $i \in C_i$ (data point i in the cluster C_i), let

$$a(i) = \frac{1}{|C_i| - 1} \sum_{j \in C_i, i \neq j} d(i, j)$$

For each data point $i \in C_i$, we now define

$$b(i) = \min_{i \neq j} \frac{1}{|C_j|} \sum_{j \in C_j} d(i, j)$$



UNSUPERVISED
LEARNING

HIERARCHICAL
AND DBSCAN

Fundamentals of Machine Learning

Agenda

Hierarchical Clustering

DBSCAN Clustering

Clustering

HIERARCHICAL CLUSTERING

Hierarchical Clustering

- **Hierarchical clustering** is characterized by the development of a hierarchy or tree-like structure.
 - **Agglomerative clustering** starts with each object in a separate cluster. Clusters are formed by grouping objects into bigger and bigger clusters.
 - **Divisive clustering** starts with all the objects grouped in a single cluster. Clusters are divided or split until each object is in a separate cluster.
- Agglomerative methods are commonly used in marketing research. They consist of linkage methods, variance methods, and centroid methods.

Hierarchical Clustering

Cluster similarity or dissimilarity

- Distance metric

- Euclidean distance $d(p, q) = \sqrt{\sum_{i=1}^n (q_i - p_i)^2}$

- Manhattan distance $d(p, q) = \sum_{i=1}^n |q_i - p_i|$

- Linkage criteria

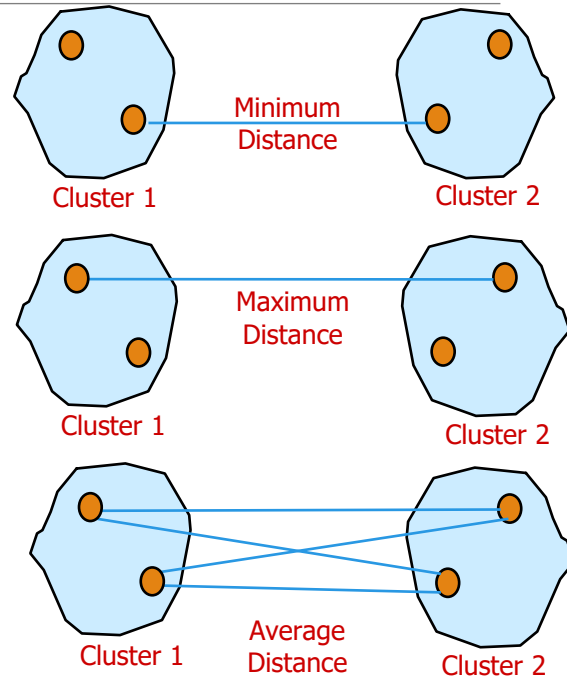
- Single linkage
 - Complete linkage
 - Average linkage

Hierarchical Clustering

Hierarchical Agglomerative Clustering

Linkage Method

- The **single linkage** method is based on minimum distance, or the nearest neighbor rule.
- The **complete linkage** method is based on the maximum distance or the furthest neighbor approach.
- The **average linkage** method the distance between two clusters is defined as the average of the distances between all pairs of objects



Hierarchical Clustering

Agglomerative Clustering Algorithm

Basic algorithm

- Compute the distance matrix between the input data points.
- Let each data point be a cluster
- **Repeat**

Merge the two closest clusters

Update the distance matrix

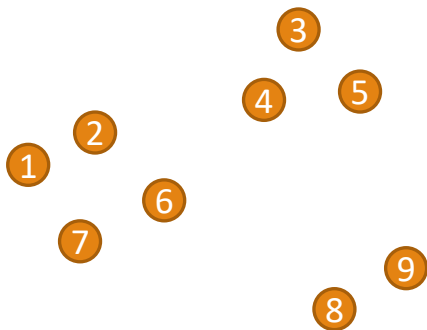
Until only a single cluster remains

Hierarchical Clustering

Agglomerative Clustering Example

Input/Initial setting

- Start with clusters of individual points and a distance matrix



	p1	p2	p3	p4	p5	...
p1						
p2						
p3						
p4						
p5						
⋮						
⋮						
⋮						

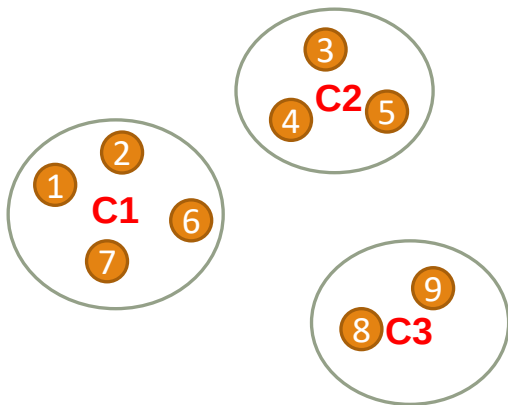
Distance Matrix

Hierarchical Clustering

Agglomerative Clustering Example

Intermediate State

- After some merging steps, we have some clusters



	C1	C2	C3
C1			
C2			
C3			

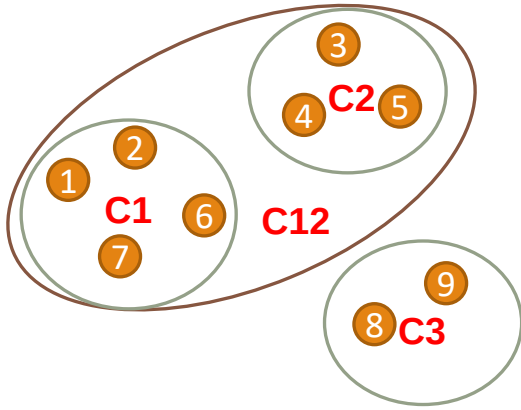
Distance Matrix

Hierarchical Clustering

Agglomerative Clustering Example

Intermediate State

- Merge the two closet clusters (C1 and C2) and update the distance matrix



	C1	C2	C3
C1	0.0	0.2	0.5
C2		0.0	0.4
C3			0.0

Distance Matrix



	C12	C3
C12		
C3		

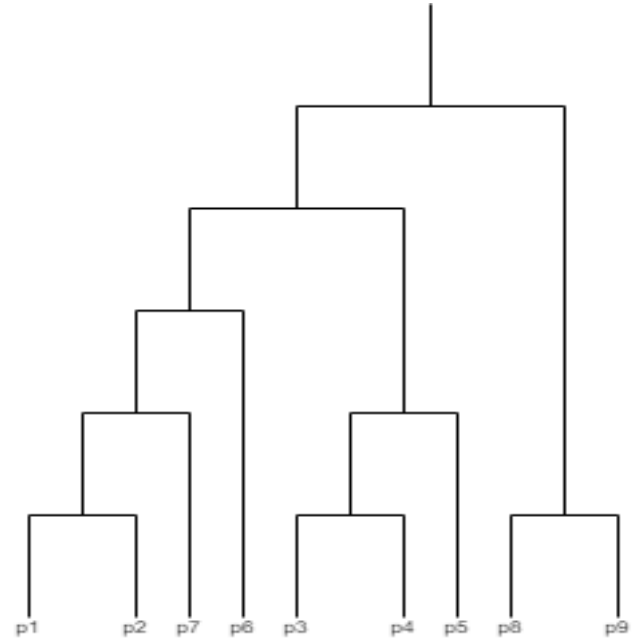
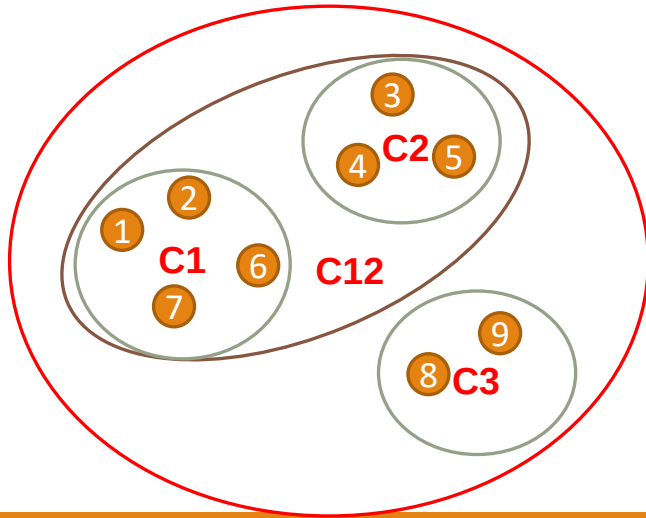
Distance Matrix

Hierarchical Clustering

Agglomerative Clustering Example

Stop

- Only a single cluster remains



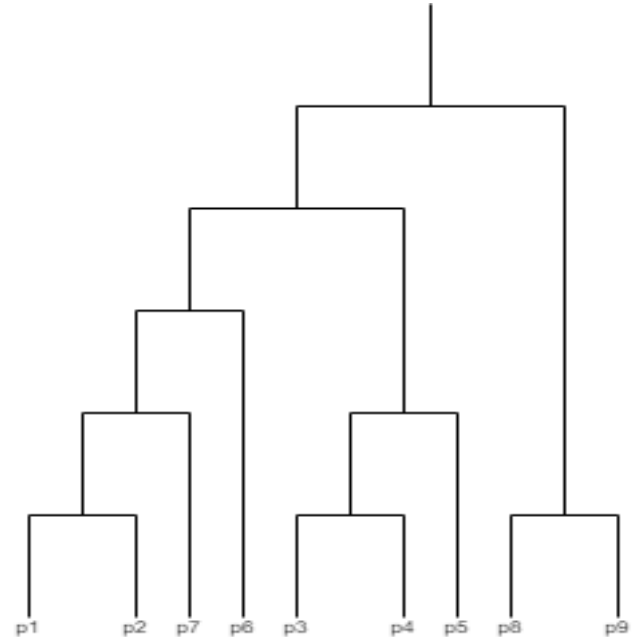
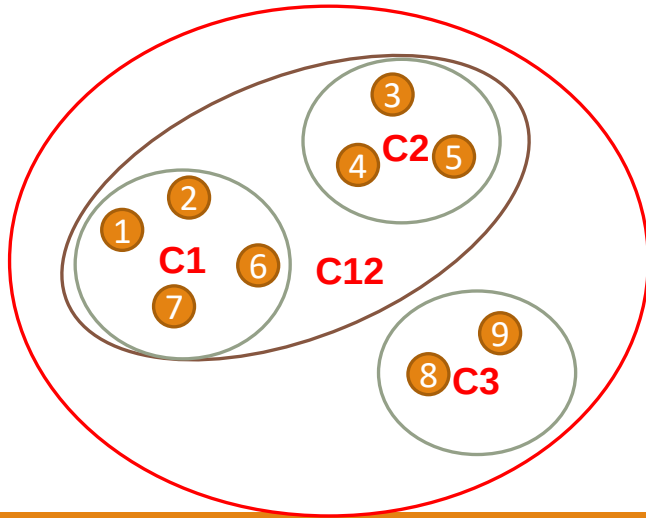
Dendrogram

Hierarchical Clustering

Agglomerative Clustering Example

Stop

- Only a single cluster remains



Dendrogram

Hierarchical Clustering

Advantages vs Disadvantages

Advantages

- Doesn't required number of clusters to be specified.
- Easy to implement
- Produces a dendrogram, which helps with understanding the data

Disadvantages

- Can never undo any previous steps throughout the algorithm
- Generally has long runtime
- Sometimes difficult to identify the number of cluster by dendrogram

Hierarchical Clustering

Agglomerative Clustering with sklearn

```
from sklearn.cluster import AgglomerativeClustering
import numpy as np
X = np.array([[1, 2], [1, 4], [1, 0], [4, 2], [4, 4], [4, 0]])
cluster = AgglomerativeClustering(n_clusters = 2, linkage = 'average',
affinity='euclidean')
clustering = cluster.fit(X)
labels = clustering.labels_
print(labels)
```


Hierarchical Clustering

PRACTICE

DBSCAN

Clustering

DBSCAN

What is DBSCAN?

- **DBSCAN is a density-based algorithm**
- DBSCAN stands for Density-Based Spatial Clustering of Applications with Noise
- Density-based Clustering locates regions of high density that are separated from one another by regions of low density

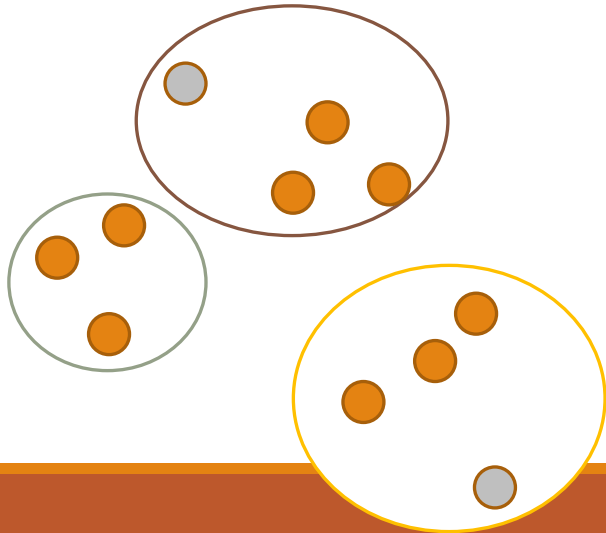
Density = number of points within a specified radius (R)

DBSCAN

K-Means vs DBSCAN?

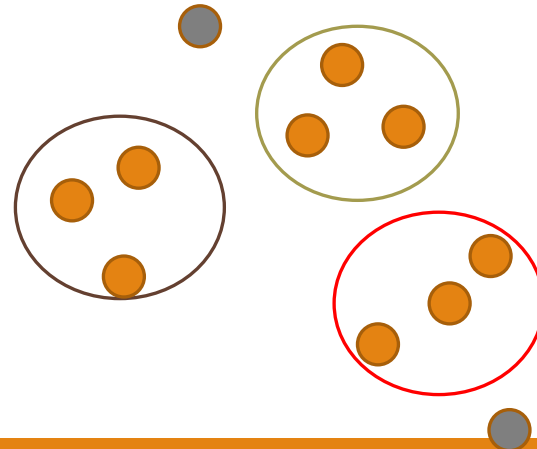
K-Means

K-means assigns all points to a cluster even if they do not belong in any



DBSCAN

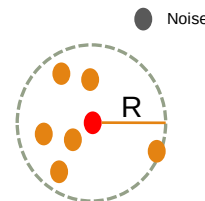
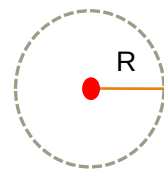
Density-based clustering locates region of **high density**, and separates outliers



DBSCAN

Radius and Min number neighbors

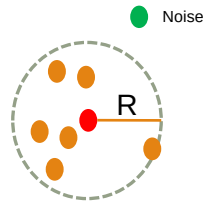
- R - Radius of neighborhood
 - Radius that if includes enough number of points within, we call it a dense area
- M – Min number of neighbors
 - The minimum number of the data points we want in a neighborhood to define a cluster



DBSCAN

Core point, border point and noise point

- A point is a **core point** if it has more than specified number of point (M) within (R)
- A **border point** has fewer than M within R, but is in the neighborhood of a core point
- A **noise point** is any point that is not a core point or border point

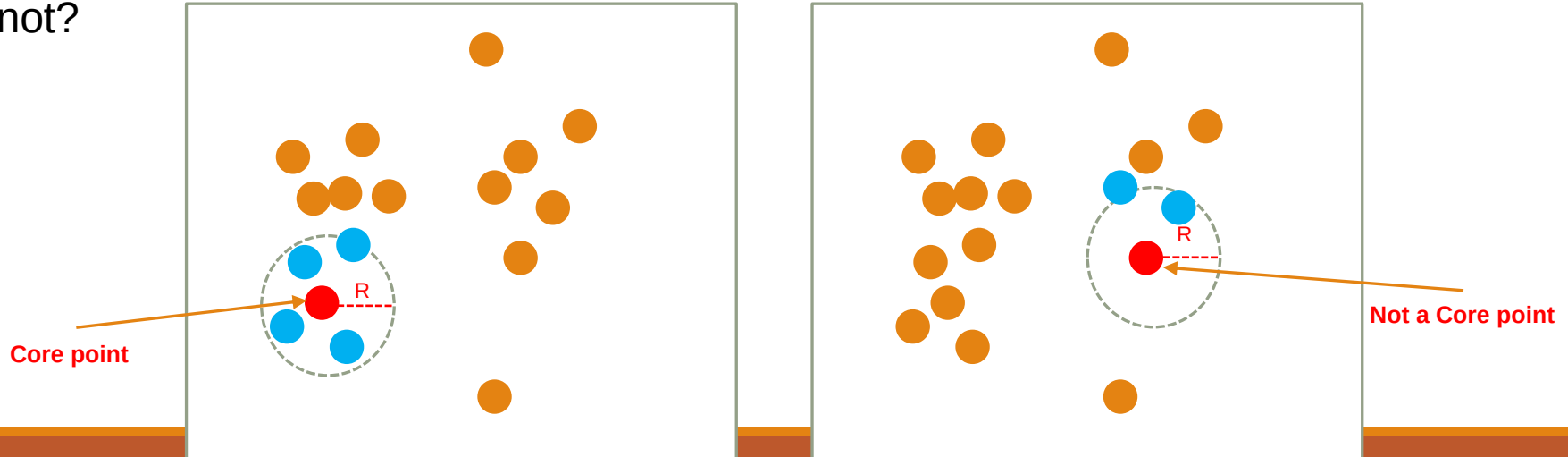


DBSCAN

How DBSCAN works

- Example: $R = 3$, $M = 4$

Randomly select a point and check if it is core point or not?

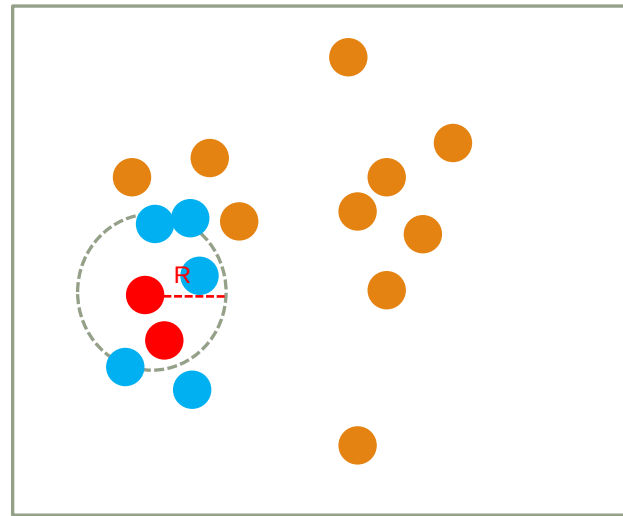


DBSCAN

How DBSCAN works

- Example: $R = 3$, $M = 4$

When determining a point as a core point, we check the current border points to see if any point is the next core point.

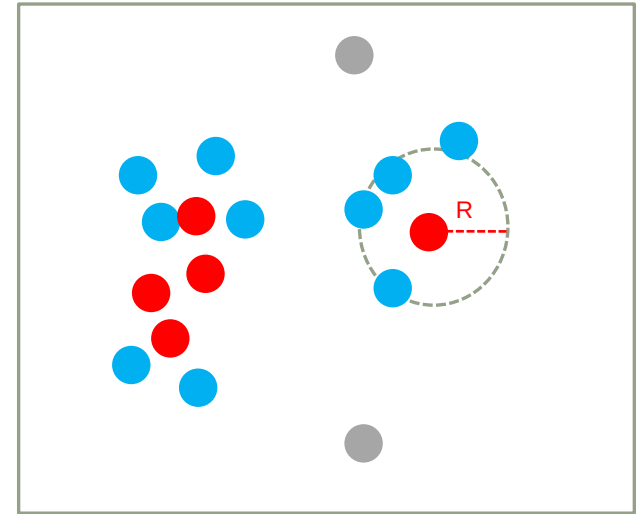


DBSCAN

How DBSCAN works

- Example: $R = 3$, $M = 4$

We continue and visit all the points in the dataset and label them as either core, border, or outlier



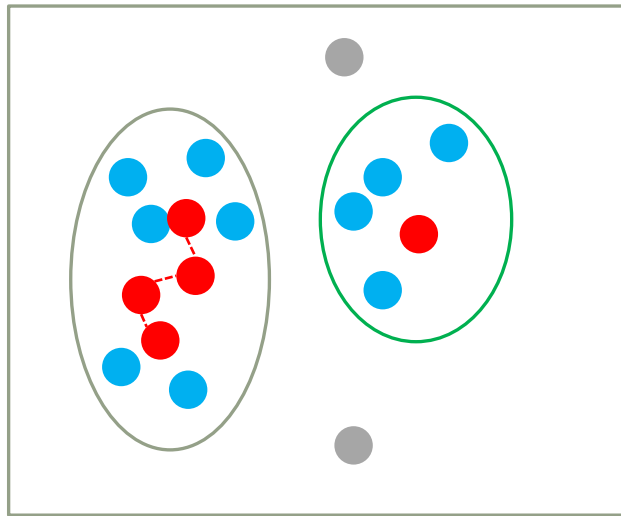
DBSCAN

How DBSCAN works

- Example: $R = 3$, $M = 4$

The next step is to connect core points that are neighbors and put them in the same cluster.

A cluster is formed as at least one core point plus all reachable core points plus all their borders



DBSCAN

Advantages vs Disadvantages

Advantages of DBSCAN:

- Arbitrarily shaped clusters
- Robust to outliers
- Does not require specification of the number of clusters

Disadvantages of DBSCAN

- Does not work very well for sparse datasets or datasets with varying density.
- Sensitive to R and M parameters.
- Not partitionable for multiprocessor systems.

DBSCAN

DBSCAN with sklearn

```
from sklearn.cluster import DBSCAN
import numpy as np
X = np.array([[1, 2], [2, 2], [2, 3], [8, 7], [8, 8], [25, 80]])
clustering = DBSCAN(eps=3, min_samples=2).fit(X)
labels = clustering.labels_
print(labels)
```

- **Eps: float, default=0.5**: The maximum distance between two samples for one to be considered as in the neighborhood of the other.
- **min_samples: int, default=5**: The number of samples (or total weight) in a neighborhood for a point to be considered as a core point.

DBSCAN

PRACTICE

Thank you
